

TeamCore

Full Step-by-Step Development Roadmap

Backend: Django + Django REST Framework + PostgreSQL

Web: Django Templates + HTML + CSS + JavaScript + Tailwind CSS

Mobile: Flutter

Target: ~160 employees initially, scalable toward 1,000+ users

Development principle: Build the backend and core workflows first, then web UI, API, Flutter, background jobs, analytics and AI.

1. Final Development Philosophy

TeamCore should be built incrementally. We should not create all models, views and templates at once. Each domain should be completed, tested and stabilized before moving to the next major domain.

Recommended internal order for each app: Model → Migration → Admin → Service/Business Logic → Permissions → URLs → Views → Forms/Serializers → Templates → Tests.

This keeps the project maintainable and makes debugging much easier as the system grows.

2. Final Django App List

#	Django App	Main responsibility
1	core	Shared utilities, base infrastructure and common logic.
2	accounts	User, roles, authentication and employee identity.
3	departments	Departments, teams and organization hierarchy.
4	projects	Projects, members and milestones.
5	tasks	Tasks, comments, dependencies and task workflow.
6	qa	QA workflow and bug management.
7	productivity	Time tracking, workload and productivity indicators.
8	attendance	Attendance, leave and holidays.
9	notifications	In-app, email and push notification records.
10	reports	Dashboards, reports and analytics.
11	files	Controlled file and attachment management.
12	audit_logs	Audit trail and critical activity history.

Total: 12 Django apps. This is enough for a clean first architecture without creating unnecessary microservices or dozens of tiny apps.

3. Recommended Project Structure

```
TaskCore-ERP/ ├── backend/ | ├── manage.py | ├── config/ | ├── core/ | ├── accounts/ | ├── departments/ | ├── projects/ | ├── tasks/ | ├── qa/ | ├── productivity/ | ├── attendance/ | ├── notifications/ | ├── reports/ | ├── files/ | ├── audit_logs/ | ├── mobile/ | ├── flutter_app/ | ├── docs/ | └── README.md
```

The current backend folder name can remain as-is. The roadmap is about architecture and implementation order, not forcing a folder rename.

4. PHASE 0 — Architecture & Requirements Freeze

Goal: finalize the product before writing large amounts of code.

- Confirm the 7 roles: ADMIN, HR, PROJECT_MANAGER, COT_HOD, EMPLOYEE, CLIENT, QA.
- Confirm organization hierarchy: Department → Team → Employee.
- Confirm project ownership and membership rules.
- Confirm task workflow and QA/rework workflow.
- Confirm what each role can see and modify.
- Confirm MVP scope versus future features.

Deliverable: one stable functional specification and permission matrix.

5. PHASE 1 — Django Foundation

Goal: create a clean technical foundation before business models.

- Create Django project/config package.
- Create virtual environment and requirements.txt.
- Configure environment variables.
- Connect PostgreSQL.
- Configure installed apps and middleware.
- Configure templates, static and media.
- Configure timezone, language and production-safe settings.
- Create .gitignore and development settings strategy.

Do not build: dashboards or complex UI yet.

6. PHASE 2 — Core App

Goal: create shared infrastructure.

- Common model base fields if required.
- Shared utilities and constants.
- Common response/error helpers where appropriate.
- Health-check endpoint.
- Base template infrastructure can be prepared later.

Keep core small. Do not turn it into a dumping ground for unrelated business logic.

7. PHASE 3 — Accounts App

This is the first major business app and should be completed before the others.

User model should support the existing 7-role structure:

ADMIN • HR • PROJECT_MANAGER • COT_HOD • EMPLOYEE • CLIENT • QA

Core user data: employee ID, first/last name, email, phone, avatar, role, department, team, designation, manager, active status, created_at, updated_at.

- Finalize custom User model.
- Set AUTH_USER_MODEL correctly.
- Create and test migrations.
- Create UserAdmin.
- Create superuser.
- Implement login/logout.
- Implement role-aware access foundation.
- Add account status handling.
- Write model and authentication tests.

Deliverable: reliable authentication and a correct User model before creating dependent models.

8. PHASE 4 — Departments App

Models: Department, Team.

- Create Department model.
- Create Team model linked to Department.
- Connect User to Department and Team.
- Support manager/HOD relationships.
- Admin management.
- CRUD views.
- Permission checks.

- Basic department/team templates.
- Tests.

Final relationship: Department → Team → Employee.

9. PHASE 5 — Projects App

Models: Project, ProjectMember, Milestone.

- Project CRUD.
- Project manager relationship.
- Client relationship where applicable.
- Project members.
- Milestones.
- Project status and priority.
- Start/deadline.
- Project dashboard foundation.
- Project permission rules.
- Tests.

10. PHASE 6 — Tasks App

This is the heart of TeamCore.

Initial models: Task, TaskDependency, TaskComment.

Later supporting models: TaskAttachment, TaskActivity if needed.

Task fields: project, parent task, creator, assignee, reviewer, QA owner, title, description, priority, status, progress, start date, due date, estimated hours, actual hours.

- Task CRUD.
- Task assignment.
- Task list and detail.
- Task filtering.
- Task priority.
- Deadline management.
- Progress update.
- Comments.
- Dependencies.
- Subtasks/parent tasks.
- Activity history.

11. PHASE 7 — Task Workflow & Services

Do not place all workflow logic directly inside views.

Recommended service functions:

```
assign_task() change_task_status() update_task_progress() block_task() submit_for_review() submit_for_qa() complete_task()
```

Recommended workflow:

```
BACKLOG → TODO → IN_PROGRESS → IN_REVIEW → QA → COMPLETED
```

Alternative branch: IN_PROGRESS → BLOCKED. QA failure: QA → REWORK → QA.

The service layer should validate who can perform each transition and record the activity.

12. PHASE 8 — QA & Bug App

Models: QARecord, Bug, BugComment.

- QA queue.
- Assign QA owner.
- Test result.
- PASS / FAIL.
- Rework flow.
- Bug severity and priority.
- Bug evidence.
- Bug comments.
- Reopen flow.
- QA verification.

Workflow: Developer → Review → QA → PASS → Completed OR FAIL → Rework → QA.

13. PHASE 9 — Productivity App

Models: TimeEntry and optionally WorkSession.

- Start timer.
- Pause/resume.
- Stop timer.
- Manual time entry with controlled correction.
- Estimated vs actual effort.
- Employee workload.
- Team capacity.
- Overload indicators.

Productivity should emphasize delivery, quality, blockers and workload rather than invasive surveillance.

14. PHASE 10 — Attendance App

Models: Attendance, LeaveType, LeaveRequest, Holiday.

- Check-in/check-out.
- Attendance status.
- Late/early indicators.
- Leave request.
- Manager/HOD approval.
- HR approval/policy validation.
- Holiday calendar.
- Attendance reports.

Leave flow: Employee → Manager/HOD → HR → Approved/Rejected.

15. PHASE 11 — Notifications App

Model: Notification.

- Task assigned.
- Deadline approaching.
- Task overdue.
- Task blocked.
- QA passed/failed.
- Mention notification.
- Project update.
- Leave approval/rejection.

- Read/unread state.

Start with database + web notifications. Add push/email after the core workflow is stable.

16. PHASE 12 — Files App

Model: Attachment.

- Project attachments.
- Task attachments.
- Bug/QA evidence.
- Comment attachments.
- File type validation.
- File size limits.
- Permission-based access.
- Safe production media storage.

17. PHASE 13 — Audit Logs

Model: AuditLog.

- Actor.
- Action.
- Object type.
- Object ID.
- Timestamp.
- Before/after data where appropriate.
- Search by user/project/task/date.

Example: User A assigns Task #102 → User B starts task → QA rejects → task moves to REWORK → QA approves.

18. PHASE 14 — Admin Panel

Once the main models exist, make Django Admin useful for internal administration.

- User administration.
- Departments and teams.
- Projects and milestones.
- Tasks and QA.
- Attendance and leave.
- Notifications.
- Attachments.
- Audit logs.

Admin is for controlled administration; normal employees should use the custom web UI.

19. PHASE 15 — Permissions

Implement permission rules after the core models and workflows exist so permissions match real business operations.

Role	Main access
ADMIN	Full system access.
HR	Employees, departments, attendance, leave and HR reports.
PROJECT_MANAGER	Own projects, teams, tasks, assignment and project reports.

Role	Main access
COT / HOD	Department/team visibility and approvals.
EMPLOYEE	Own permitted tasks, projects, comments, attendance and leave.
QA	QA queue, bugs, testing and verification.
CLIENT	Only permitted client projects and approved information.

Always enforce authorization server-side. Hiding a button is not security.

20. PHASE 16 — URLs & Navigation

Create clean URL namespaces after each domain is stable.

`/accounts/ /departments/ /projects/ /tasks/ /qa/ /productivity/ /attendance/ /notifications/ /reports/ /files/`

Use `app_name` and namespaces to prevent route collisions.

21. PHASE 17 — Views

Use standard CRUD views where possible and service calls for business operations.

Typical views:

- ListView — lists and filters.
- DetailView — detailed record.
- CreateView — creation form.
- UpdateView — editing.
- DeleteView — controlled deletion.
- DashboardView — summarized information.
- Action views — assign, approve, status transition, QA actions.

22. PHASE 18 — Forms & Serializers

Django Forms: web interface validation.

DRF Serializers: API validation and representation.

Do not duplicate complex business rules in every form and serializer. Shared business rules belong in services/domain logic.

23. PHASE 19 — Templates & Tailwind UI

Only after the backend workflows are stable should the complete UI be built.

`templates/ |— base.html |— includes/ | |— navbar.html | |— sidebar.html | |— messages.html | |— footer.html |— accounts/ |— departments/ |— projects/ |— tasks/ |— qa/ |— productivity/ |— attendance/ |— notifications/ |— reports/`

- Responsive layout.
- Dashboard cards.
- Tables.
- Filters.
- Forms.
- Modal dialogs.
- Status badges.
- Charts.
- Pagination.
- Empty states.
- Permission-aware action buttons.

24. PHASE 20 — Dashboards

Company dashboard: employees, projects, active tasks, completed today, overdue, blocked and at-risk projects.

Employee dashboard: my tasks, deadlines, progress, blocked items, notifications, attendance and leave.

PM dashboard: project health, milestones, workload, overdue, blockers, QA and delivery forecast.

HOD dashboard: department workload, team trends, project risks and approvals.

QA dashboard: test queue, bugs, severity, pass/fail, reopens and release readiness.

25. PHASE 21 — REST API

Only after the Django web workflows are reliable, build the API layer for Flutter.

Base: /api/v1/

Domain	Example
Auth	/api/v1/auth/login, refresh, logout
Employees	/api/v1/employees
Projects	/api/v1/projects
Tasks	/api/v1/tasks
Task actions	/api/v1/tasks/{id}/assign, progress, status
QA	/api/v1/qa/queue
Notifications	/api/v1/notifications
Reports	/api/v1/reports/project-health

Use consistent authentication, pagination, filtering, validation, error responses and permissions.

26. PHASE 22 — Flutter Mobile App

Flutter should consume the same Django REST API.

Version 1 — Employee: Login, dashboard, my tasks, task detail, progress, comments, files, notifications, attendance, leave, profile.

Version 2 — Manager: projects, team, task board, assignment, approvals, reports.

Version 3 — QA: QA queue, test detail, bugs, re-test and verification.

Use secure token storage, pagination, push notifications and selected offline-friendly caching.

27. PHASE 23 — Redis & Celery

Add background processing only after core features work correctly.

Django | — PostgreSQL | — Redis | Celery | — Notifications | — Deadline reminders | — Overdue checks | — Reports | — Scheduled jobs

This prevents slow background operations from blocking normal web/API requests.

28. PHASE 24 — Reports & Analytics

- Project health.
- Task completion.
- Overdue analysis.
- Blocked duration.
- Workload/capacity.

- Estimated vs actual effort.
- QA pass/rework.
- Bug trends.
- Attendance/leave.
- Department/team trends.

Exports can be added for authorized users: CSV, XLSX and PDF.

29. PHASE 25 — AI Features

AI comes after reliable operational data exists.

- AI task estimation.
- Project risk prediction.
- Suggested task assignment.
- AI weekly project summary.
- Anomaly detection.
- Natural-language management assistant.

AI should remain an advisory layer. Human managers retain authority over staffing, assignment and performance decisions.

30. PHASE 26 — Testing

Testing level	What to test
Unit	Models, services, permissions and business rules.
API	Authentication, validation, CRUD and authorization.
Workflow	Task status, QA/rework and approvals.
Security	Permissions, object access, upload and authentication.
Concurrency	Assignment and state-change race conditions.
Flutter	Core mobile workflows and API integration.
Performance	Dashboards, large task lists and reports.
Recovery	Database backup and restore.

31. PHASE 27 — Performance & 1,000+ User Readiness

- Add database indexes based on real query patterns.
- Paginate every large list.
- Use `select_related/prefetch_related` to avoid N+1 queries.
- Use Redis caching where useful.
- Move heavy jobs to Celery.
- Optimize dashboard queries.
- Use object/media storage for files.
- Monitor DB load, API latency, errors and queue health.
- Load test before major rollout.

32. PHASE 28 — Security Hardening

- HTTPS.
- Secure Django settings.

- Environment-based secrets.
- Strong authentication.
- RBAC and object-level authorization.
- CSRF protection.
- API throttling.
- Safe file uploads.
- Audit logs.
- Database backups.
- Dependency/security updates.

33. PHASE 29 — Deployment

Development → Staging → Production

Production stack can include Django, Gunicorn, Nginx/reverse proxy, PostgreSQL, Redis, Celery worker/beat, secure media storage and monitoring.

- Separate production credentials.
- Automated deployment when ready.
- Database migration strategy.
- Backup before risky changes.
- Rollback procedure.
- Application/error monitoring.

34. Exact Build Order — Short Version

01. Project setup 02. PostgreSQL 03. core 04. accounts 05. departments 06. projects 07. tasks 08. task workflow/services 09. QA + bugs 10. productivity 11. attendance 12. notifications 13. files 14. audit_logs 15. admin 16. permissions 17. URLs 18. forms/serializers 19. views 20. templates + Tailwind 21. dashboards 22. REST API 23. Flutter 24. Redis + Celery 25. reports/analytics 26. AI 27. testing 28. security hardening 29. deployment

This is the recommended sequence for the actual development work.

35. Per-App Completion Checklist

For every app, follow this checklist before moving forward:

- 1. Define requirements.
- 2. Design models.
- 3. Write models.
- 4. makemigrations.
- 5. migrate.
- 6. Configure admin.py.
- 7. Create service/business logic.
- 8. Create permissions.
- 9. Create URLs.
- 10. Create forms/serializers.
- 11. Create views.
- 12. Create templates.
- 13. Add navigation.
- 14. Test CRUD.
- 15. Test permissions.
- 16. Test edge cases.
- 17. Refactor before moving to the next domain.

36. First Sprint — What We Should Do Now

Do NOT start Flutter yet.

Our immediate sprint should be:

- Finalize Django project settings.
- Connect PostgreSQL.
- Finalize accounts User model.
- Finalize all 7 roles.
- Create accounts/admin.py.
- Create migrations and migrate.
- Create superuser.
- Verify Django Admin.
- Implement login/logout.
- Test User creation and role behavior.

After this is stable, we move to departments.

37. Definition of Done

A feature is not considered complete merely because the model exists.

For TeamCore, a domain is complete when its model, migration, admin, business rules, permissions, URLs, views, forms/serializers, templates, tests and error handling are working together.

This approach will make the final product much easier to maintain when the company grows from the current size toward 1,000+ users.

End of TeamCore Full Step-by-Step Development Roadmap.